CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

Model Evaluation - Classification

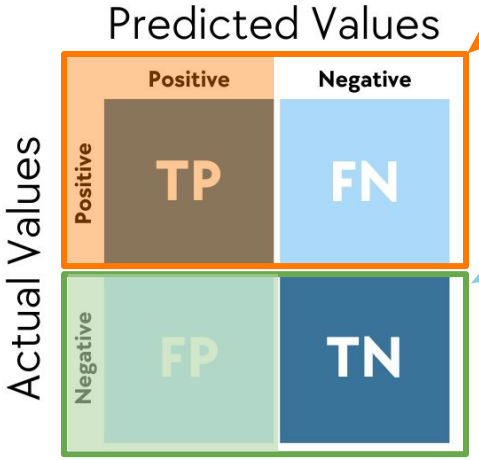
Chapter 3

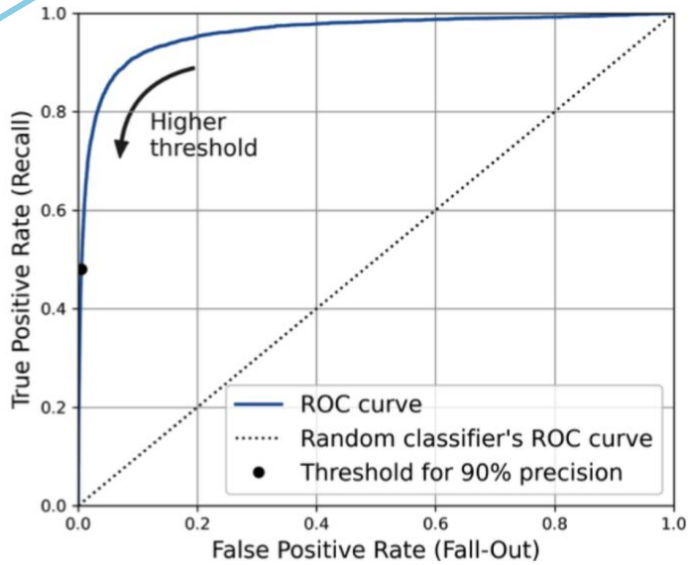
- Binary Classification
- Confusion Matrices
  - Accuracy / Error
  - Confusion Matrix
  - Precision, Recall, F1 Score
  - **ROC / AUC**
- **Multi-class Evaluation**

Receiver Operating Characteristic (ROC) Curve

Similar to the precision/recall curve, but

- instead of plotting precision versus recall
- ROC curve plots **True Positive Rate (TPR/recall)** against **False Positive Rate (FPR)**





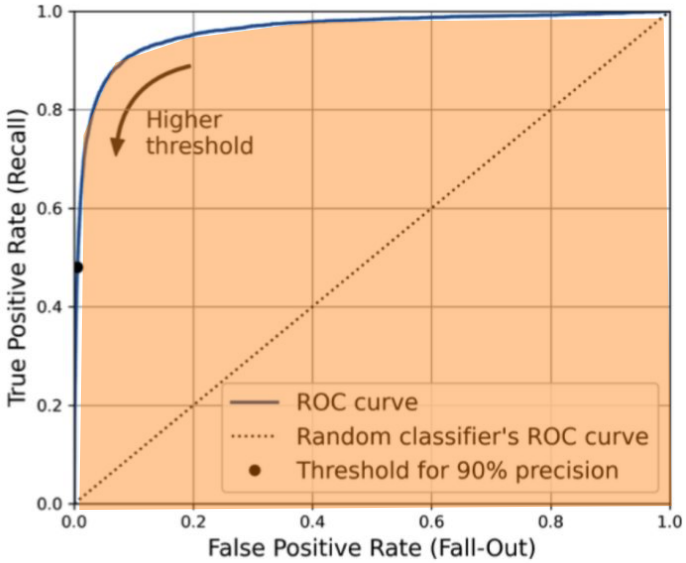
Receiver Operating Characteristic (ROC) Curve

A **good classifier** should be located as close as possible to the upper-left corner, while a random classifier would be along the main diagonal

- Useful for comparing the relative performance among different classifiers

The **area under the ROC curve** (**AUC**) provides information on which classifier is better on average

- If the model is perfect, its area would be 1
  - random guess = 0.5



From Binary to Multiclass Classification

- Binary classifier: distinguish between two classes (Positive and Negative)
- Multiclass classifier: distinguish between more than two classes

Two common strategies:

- **One-versus-All (OvA)** (i.e., One-versus-Rest)
  - Each class competes against all other classes together



Available Classes



From Binary to Multiclass Classification

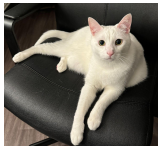
- Binary classifier: distinguish between two classes (Positive and Negative)
- Multiclass classifier: distinguish between more than two classes

Two common strategies:

- One-versus-One (OvO)
  - Every class competes directly against every other class

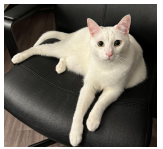


Available Classes



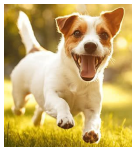
vs.





vs.





vs.



Multilabel Classification

Assigns multiple, non-exclusive labels to a single data instance at the same time



[Dog='True', Cat='True', Bird='False', Horse='False']

Linear Regression

Chapter 4

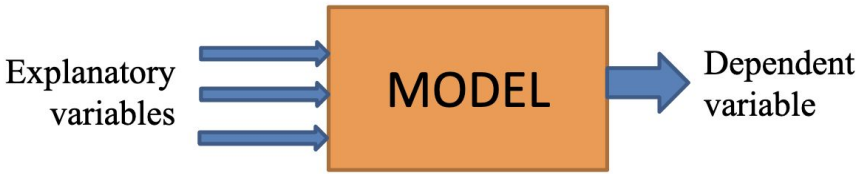
- Simple vs. Multiple
- Regression Performance Measure
- Gradient Descent
  - Batch Gradient Descent
  - Stochastic Gradient Descent
  - Mini-Batch Gradient Descent
- Polynomial Regression
- Learning Curves
- Regularized Linear Models
  - Ridge vs. Lasso vs. Elastic Net
- Logistic Regression

Regression Model

- Relationship between one dependent variable and **explanatory/independent variable(s)**
- Dependent variable is **continuous**
- One equation describes the relationship

Steps to modeling:

- Select the model family
  - Specify the generic pattern of relationship between variables
  - Linear, quadratic, …
- **Fit** the model to the data
  - Give values to the model parameters
  - Identify the “best” model from the family of models



Linear Regression

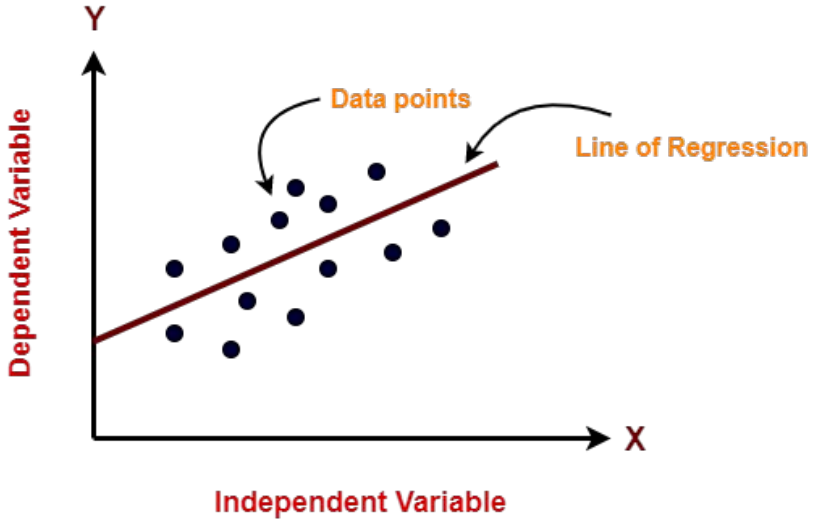
- A statistical and machine learning method used to model the relationship between a dependent target variable and one or more independent explanatory features by fitting a linear equation to observed data

Mathematical speaking…

- **Simple Linear Regression**
  - With one explanatory variable

$$\hat{y} = \theta_0 + \theta_1 x$$

Price (K$) (Y)	*Size (X_1)*
375 (y_1)	1024 (x_{11})
425 (y_2)	1329 (x_{21})
…	…
465 (y_N)	1893 (x_{N1})



Linear Regression

- A statistical and machine learning method used to model the relationship between a dependent target variable and one or more independent explanatory features by fitting a linear equation to observed data

Price (K$) ($Y$)	Size (X_1)	Bedroom (X_2)	Bathroom (X_3)	Built year (X_4)
375 (y_1)	1024 (x_{11})	3 (x_{12})	2 (x_{13})	1978 (x_{14})
425 (y_2)	1329 (x_{21})	3 (x_{22})	5 (x_{23})	1992 (x_{24})
…	…	…	…	…
465 (y_N)	1893 (x_{N1})	4 (x_{N2})	4 (x_{N3})	1980 (x_{N4})

Mathematical speaking…

- **Multiple Linear Regression**
  - With multiple explanatory variable

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\begin{aligned} y_1 &= \theta_0 + \theta_1 x_{11} + \theta_2 x_{12} + \theta_3 x_{13} + \theta_4 x_{14} \\ y_2 &= \theta_0 + \theta_1 x_{21} + \theta_2 x_{22} + \theta_3 x_{23} + \theta_4 x_{24} \\ &\cdots \quad \cdots \quad \cdots \quad \cdots \quad \cdots \quad \cdots \\ y_N &= \theta_0 + \theta_1 x_{N1} + \theta_2 x_{N2} + \theta_3 x_{N3} + \theta_4 x_{N4} \end{aligned}$$

```
from sklearn.linear_model import LinearRegression

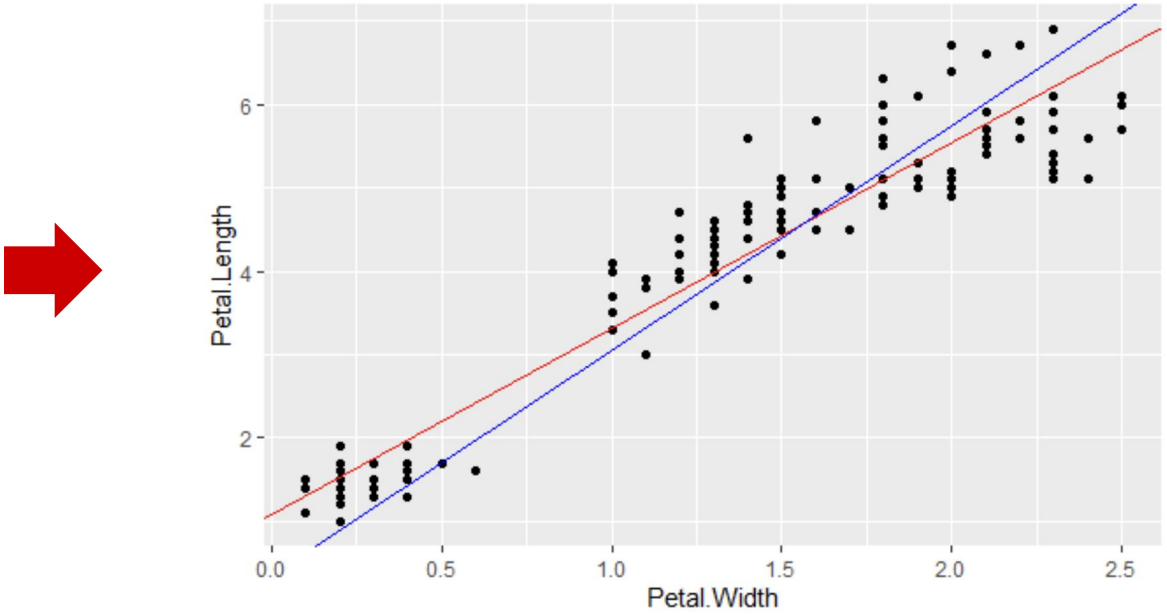
lin_reg = LinearRegression()
lin_reg.fit(X, y)
lin_reg.intercept_, lin_reg.coef_
lin_reg.predict(X_new)
```

in matrix form

$$\begin{bmatrix} 375 \\ 425 \\ \cdots \\ 465 \end{bmatrix} = \begin{bmatrix} 1 & 1024 & 3 & 2 & 1978 \\ 1 & 1329 & 3 & 5 & 1992 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 1 & 1893 & 4 & 4 & 1980 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \cdots \\ \theta_4 \end{bmatrix}$$

Which model/line is better?

Petal. Length	Petal. Width	Output
1.4	0.2	setosa
1.4	0.2	setosa
4.7	1.4	versicolor
6.0	2.5	virginica
…	…	…



Regression Performance Measure

Best Fit

- Find the weights that gives the smallest difference between *Actual Y values* $(y^{(i)})$ & *Predicted Y values* $(\hat{y}^{(i)} = \theta^T x^{(i)})$
- Differences are called **errors** or **residuals**



Why not adding all residuals together?



Regression Performance Measure

$$l(w) = \frac{1}{N}\sum_{i}(y_i - f_w(x_i))^2$$

Mean Square Error (MSE)

- A metric used to measure how well a model's predictions match the actual values
- Calculated by taking the average of the squared errors
- Commonly used as a cost function for Linear Regression

Root Mean Square Error (RMSE)

- A variant of MSE that calculates the square root of the average squared difference
- Often preferred over MSE
  - as it provides an interpretable measure of the error in the same units as the original data

$$RMSE = \sqrt{(MSE)}$$

Square Error

number of samples

real value

predicted value

$$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

sum of the errors of all samples

Sum of Squared Errors

Reference:
https://levelup.gitconnected.com/understanding-mean-squared-error-a-practical-guide-for-data-scientists-4ec7de03ac0e

Computational Complexity

in matrix form

$$\begin{bmatrix} 375 \\ 425 \\ \dots \\ 465 \end{bmatrix} = \begin{bmatrix} 1 & 1024 & 3 & 2 & 1978 \\ 1 & 1329 & 3 & 5 & 1992 \\ \dots & \dots & \dots & \dots & \dots \\ 1 & 1893 & 4 & 4 & 1980 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \dots \\ \theta_4 \end{bmatrix}$$

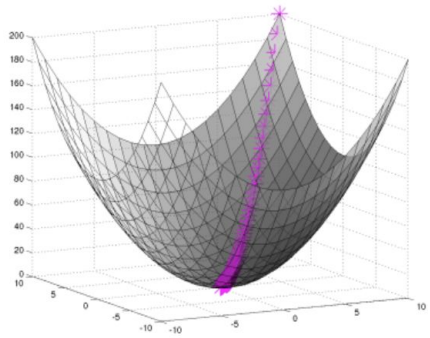
- Usually matrix operations for regression
  - i.e., Normal Equation: $\widehat{\theta} = (\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal \mathbf{y}$

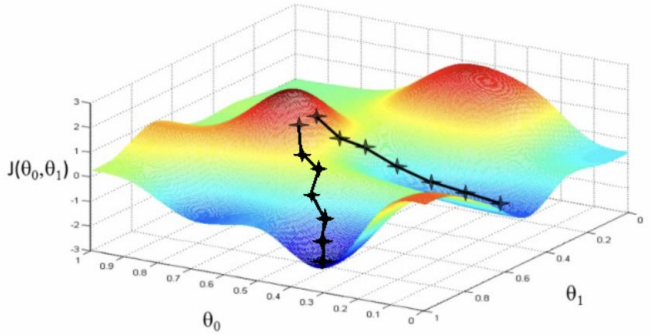
- Computationally Expensive
  - Typically $O(n^{2.4})$ to $O(n^3)$, where n is the number of features
  - For example,
    - if double the number of features, simply multiply the computation time by roughly $2^{2.4} = 5.3$ and $2^3 = 8$

Linear Regression as a *Convex Optimization* Problem

Gradient Descent (i.e., steepest gradient descent)

- An iterative optimization approach to find the minimum (or maximum) of a function by taking 1st order derivative





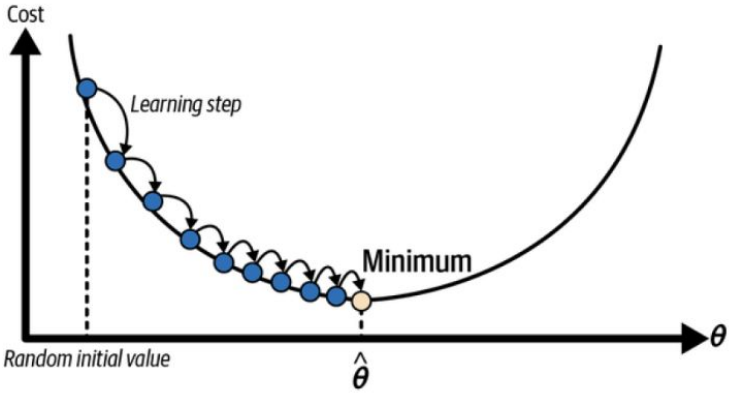
Gradient descent will reach a **global optimum** if a function has only one optimum

- Otherwise, it will reach a **local optimum**

Linear Regression as a *Convex Optimization* Problem

Algorithm wise…

I. Start with **some initial weights** e.g., ($\theta_0 = 0$, $\theta_1 = 0$)
II. **Keep changing** (θ_0, θ_1) **to reduce** $MSE(\theta_0, \theta_1; X, y)$
III. **Stop when** desirable conditions are satisfied



$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

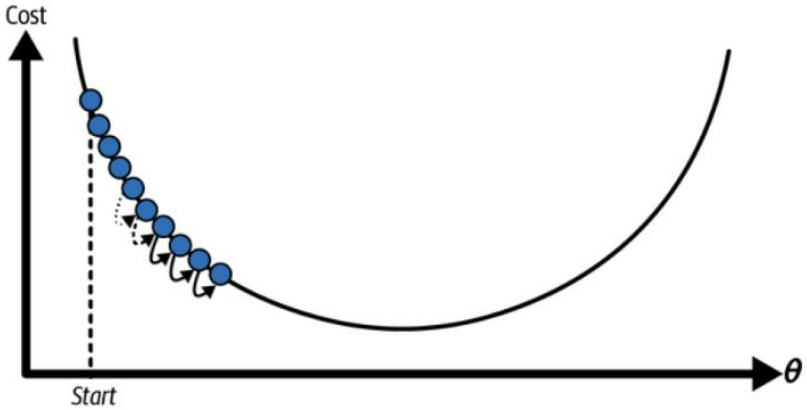
$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^\intercal (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\intercal \mathbf{x}^{(i)} - y^{(i)} \right) x_j^{(i)}$$

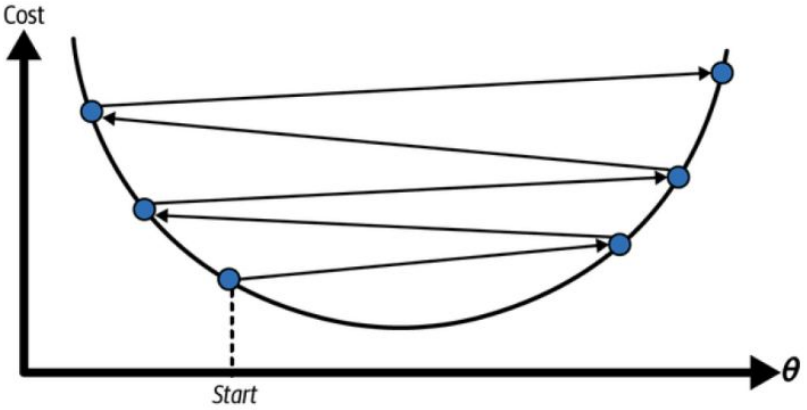
Hyperparameter η is the learning rate to control the speed of convergence

Linear Regression as a *Convex Optimization* Problem

Learning rate…



If too **small**



If too **large**

Variant of Gradient Descent Method

Batch Gradient Descent

- At each step, use ALL the training samples
- Relatively stable and consistent

$$w_j = w_j - \alpha \frac{1}{N} \sum_i (w^T x_i - y_i) x_{ij}$$

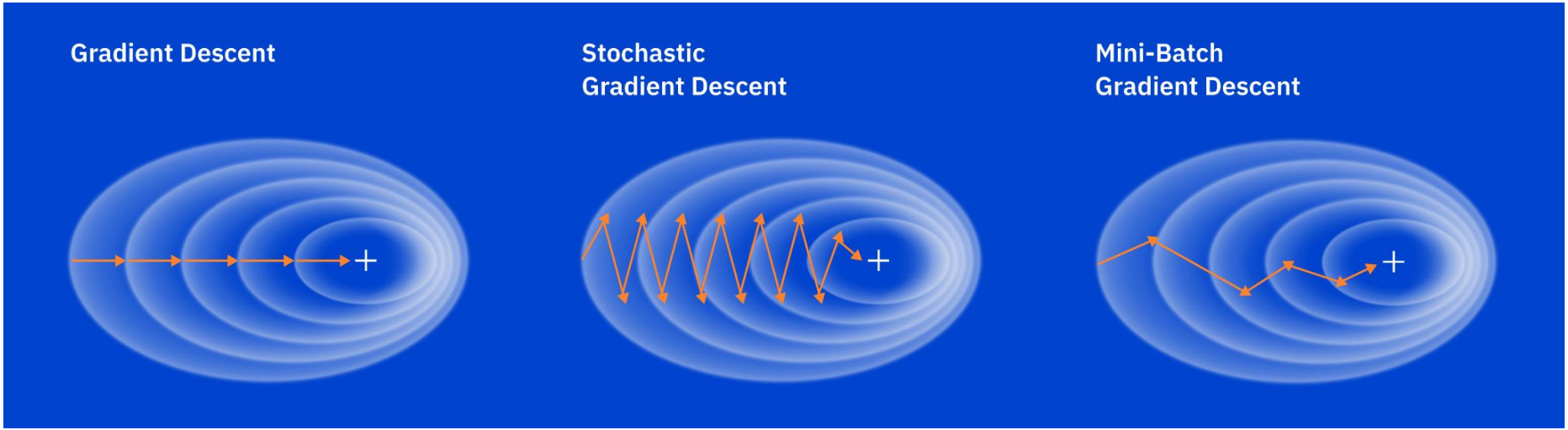
Stochastic Gradient Descent (i.e., SGD)

- Use a portion of random training samples at each iteration
- This is effective with big data or for online/adaptive learning

Mini-Batch Gradient Descent

- Combination of batch and SGD
- The training set are split into fixed-size or random batches, then update the weights for each batch
- The training set are then randomly shuffled into new batches and the process is repeated

Variant of Gradient Descent Method

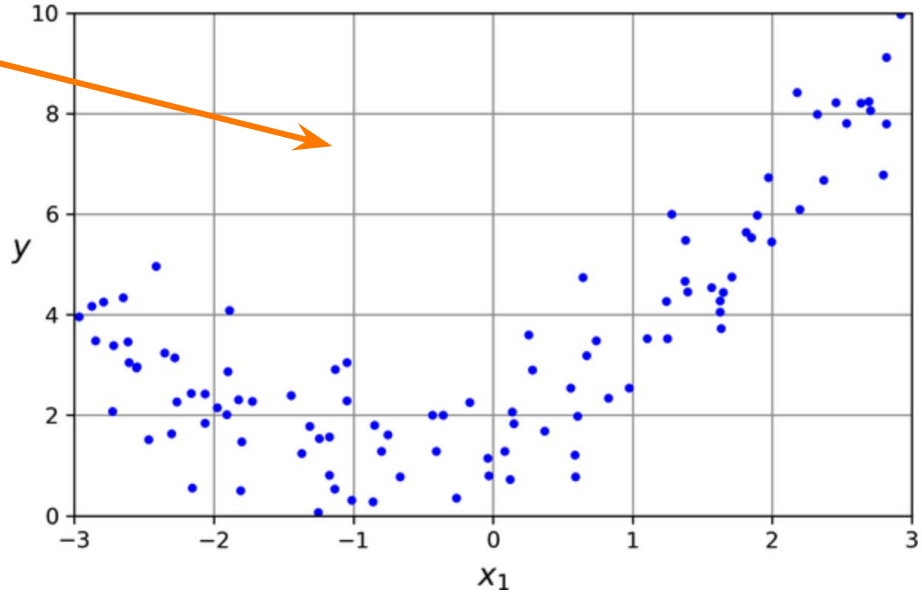


Reference:
https://www.ibm.com/think/topics/stochastic-gradient-descent

From Linear Regression to Polynomial Regression



Would a straight line fit?



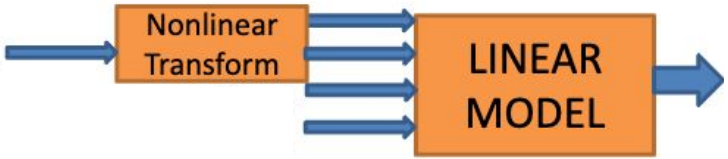
Polynomial Regression

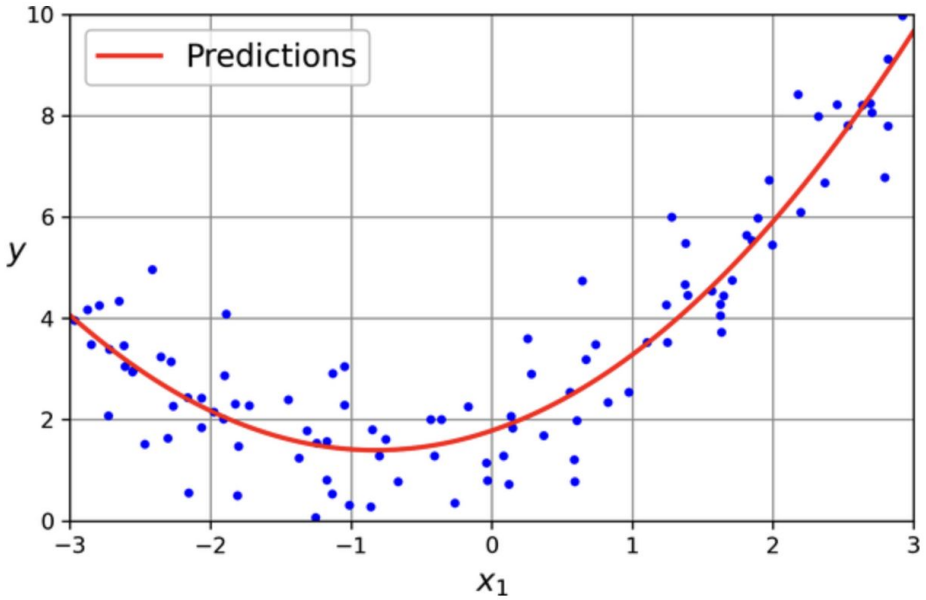
- Use a linear regression model to fit nonlinear data
- Create new independent variable from existing variables
  - Add powers of each feature

$$\theta_0 + \theta_1 x \quad \Rightarrow \quad \theta_0 + \theta_1 x + \theta_2 x^2$$

Is Polynomial Regression Really Linear?

- Polynomial Regression is still “linear” in terms of the unknown parameters (the coefficients), even though it models a non-linear relationship between the input variables and the output





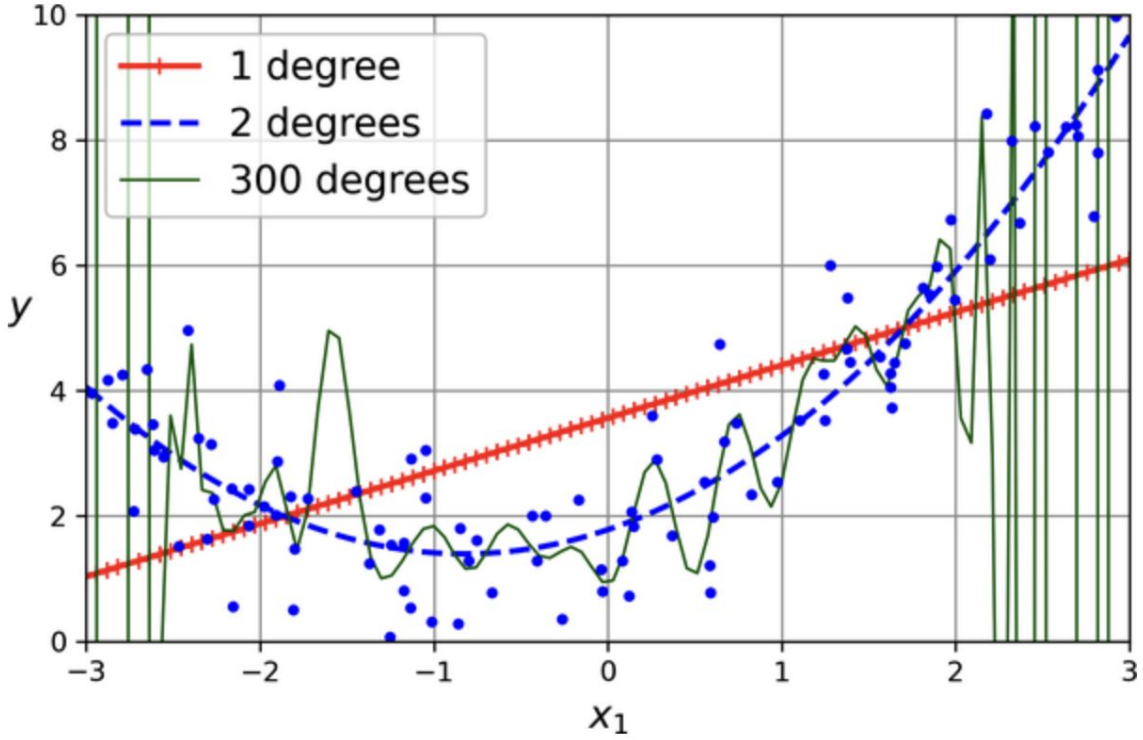
Polynomial Features

PolynomialFeatures adds all combinations of features up to the given degree

- For example, if there are two features ***a*** and ***b***
  - PolynomialFeatures with degree=3 would add:
    - $a^2, a^3, b^2, b^3, ab, a^2b, ab^2$
- Beware of the combinatorial explosion of the number of features!

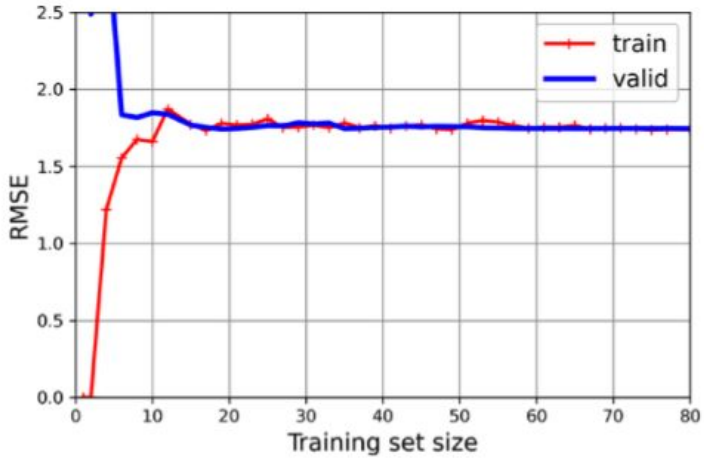
```
from sklearn.preprocessing import PolynomialFeatures
poly_features = PolynomialFeatures(degree=3, include_bias=False)
X_poly = poly_features.fit_transform(X)
```

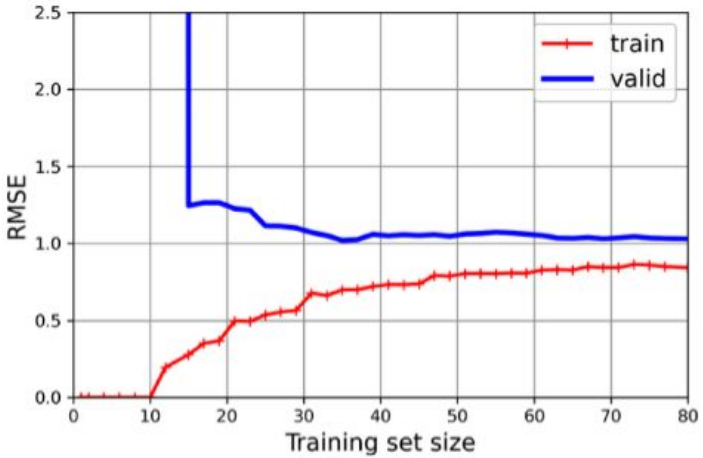
High-degree polynomial Regression



Learning Curves

- Plots of the model's training error and validation error as a function of the training iteration





Underfitting

- **Use better model**
- **Add useful features**
- **Increase polynomial degree**

Overfitting

- **Feed more training data until the validation error reaches the training error**

Bias vs. *Variance* vs. *Irreducible Error*

A model's generalization error can be expressed as the sum of three very different errors:

Bias

- Due to *wrong assumptions* (i.e., assuming data is linear when it is actually quadratic)
- A high-bias model is most likely to underfit the training data

Variance

- Due to the model's excessive sensitivity to *small variations in the training data*
- A model with many degrees of freedom (i.e., a high-degree polynomial model) is likely to have high variance and thus overfit the training data

Irreducible error

- Due to the *noisiness* of the data itself
- The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers)

Bias/Variance Trade-Off

Increasing a model's complexity will typically increase its variance and reduce its bias

- Conversely, reducing a model's complexity increases its bias and reduces its variance

Simply speaking…

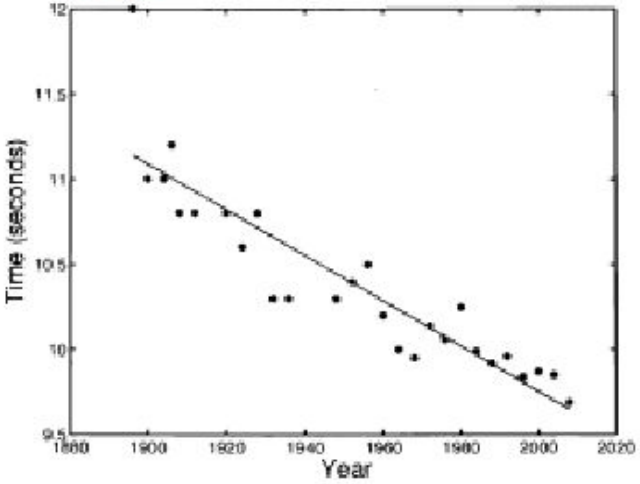
- *Highly complex model* = **low bias** + **high variance**
- *Very simple model* = **high bias** + **low variance**

Tradeoff is to determine the optimal model complexity such that it is able to generalize well without overfitting

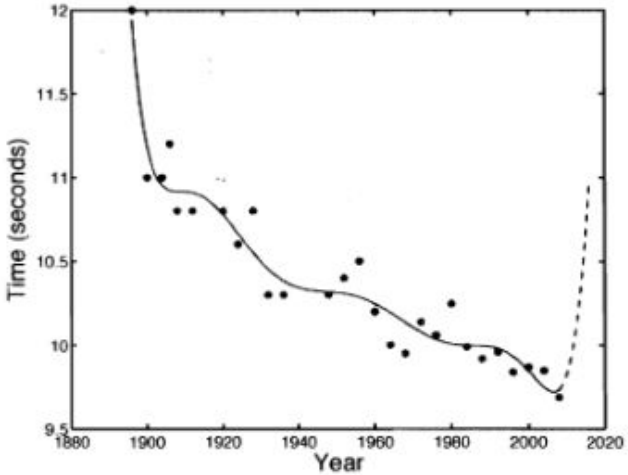
- This is a challenging task!

Complex Models

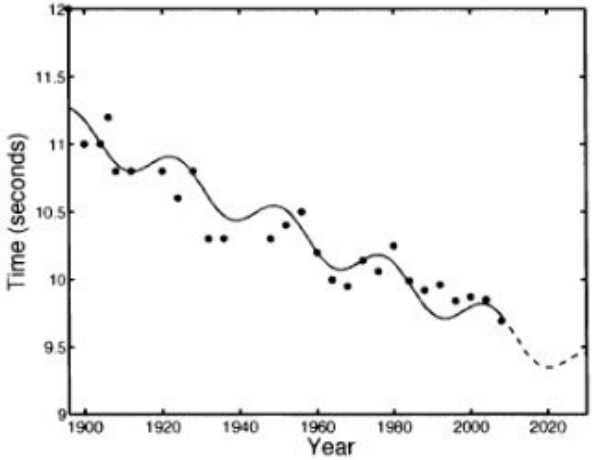
(a) A linear model with *loss* = 1.358



(b) An 8th order polynomial model with *loss* = 0.459



(c) A customized model with *loss* = 1.1037



What's going on with higher order models?

- Some predictions may be very inaccurate with small deviations in the new examples

Regularizing Models

$$l(w) = \frac{1}{N}\sum_{i}(y_i - f_w(x_i))^2$$ (**note**: this is a loss function)

Mathematically speaking, regularizing a model means…

- $l(w) + \alpha R(w)$
  - where $\alpha R(w)$ is a regularized term (i.e., penalty term / shrinkage factor)
  - α can be any real values > 0; w are shrinking coefficients

The role of the tuning parameter α

- If α = 0, linear regression (no regularization)
- The larger α, more aggressively penalization is
  - The coefficient is close to zero, resulting in a simpler model

α should be chosen wisely (using cross-validation) as it controls the tradeoff

Regularizing Models - *Three ways*

Ridge Regression

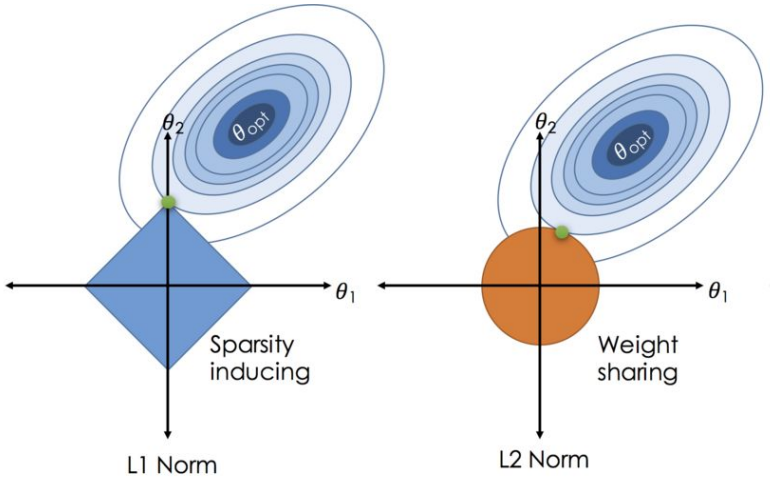
$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$$

- Also known as *L2 regularization*
- Can shrink the coefficients asymptotically close to zero, but most features still remain

LASSO Regression (i.e., *Least Absolute Shrinkage and Selection Operator*)

- Also known as *L1 regularization*
- Can shrink them ALL the way to zero,
  - Eliminating the less important features
    - Perform a form of **feature selection**

$$J(\theta) = \text{MSE}(\theta) + 2\alpha\sum_{i=1}^{n}|\theta_i|$$



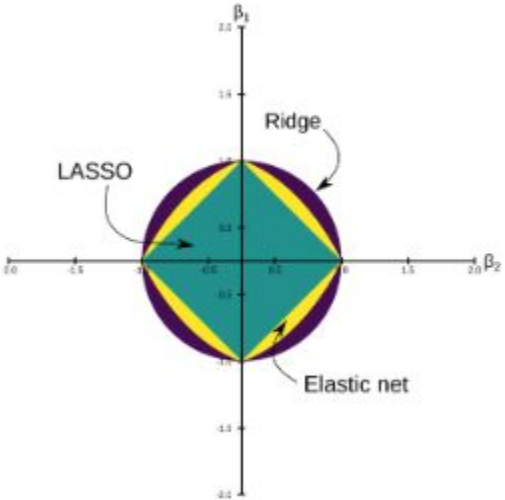
Regularizing Models - *Three ways*

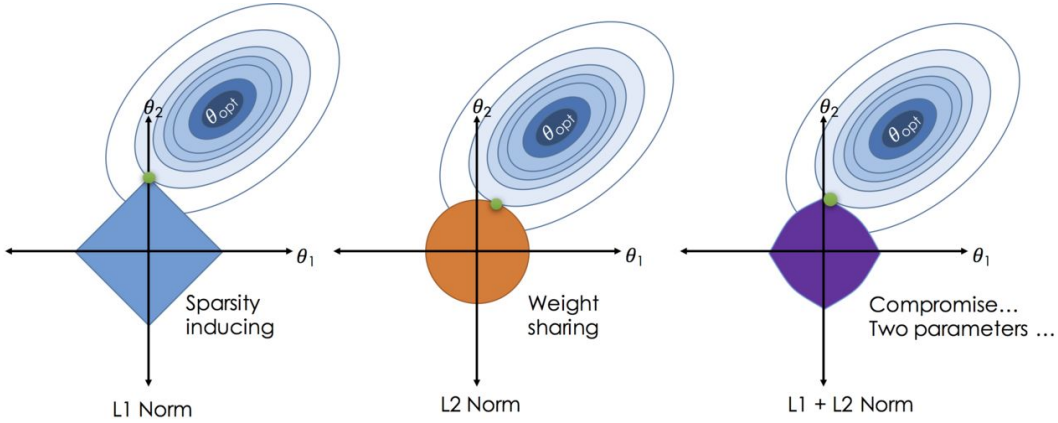
Lasso **Ridge**

$$J(\theta) = \text{MSE}(\theta) + r\left(2\alpha\sum_{i=1}^{n}|\theta_i|\right) + (1-r)\left(\frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2\right)$$

Elastic Net Regression

- Middle ground between Lasso and Ridge
- Can be particularly useful when we have many features, especially when some of them are strongly related to each other





Reference:
https://www.linkedin.com/pulse/regularization-episode-1-amr-mahmoud/

Regularizing Models - *Scikit-Learn*

```
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
from sklearn.linear_model import Lasso

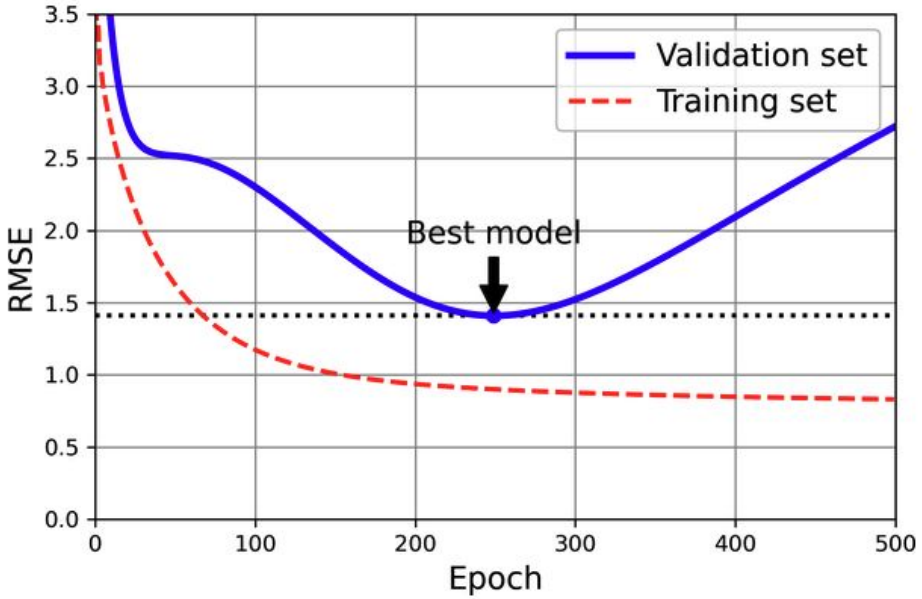
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
lasso_reg.predict([[1.5]])
```

```
from sklearn.linear_model import ElasticNet

elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
elastic_net.predict([[1.5]])
```

Regularizing Models - *Early Stopping*

- A very different way to **regularize iterative learning algorithms** such as gradient descent is to stop training as soon as the validation error reaches a **minimum**



With stochastic and mini-batch gradient descent, the curves are not so smooth, and it may be hard to know whether you have reached the minimum or not.

- One solution is to stop only after the validation error has been above the minimum for some time
- Then roll back the model parameters to the point where the validation error was at a minimum.

From Regression Back to Classification

Logistic Regression

- Modify linear regression to do classification
- A supervised machine learning algorithm used for classification problems
  - Mostly binary classifier

Model estimates the **probability** that an instance belongs to a class

- If the estimated probability > 0.5, then predict the instance belongs to that class (positive)
  - Otherwise (p < 0.5), predict that it does not belong (negative)

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Logistic regression model prediction using a 50% threshold probability

Logistic Regression

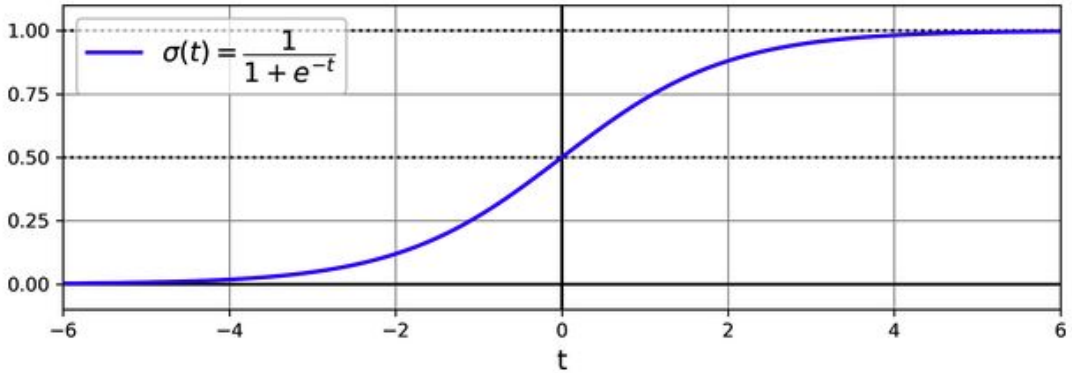
How?

- Model computes…
  - a weighted sum of the input features
  - Followed by the **logistic** of this weighted sum

Mathematically speaking…

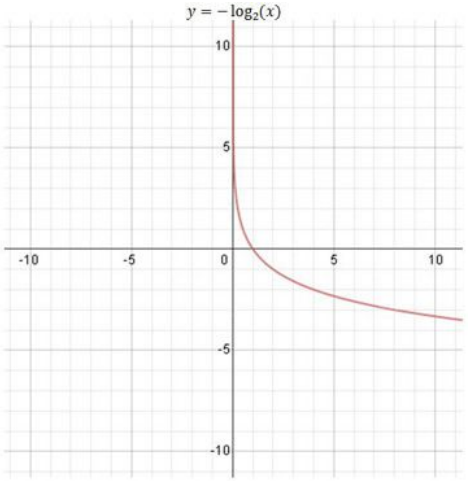
- **Logistic function**

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



A *sigmoid function* that outputs a number between 0 and 1 for any input

Training Logistic Regression - *cost function*



- Log-loss for <u>one instance</u>:

$$c(\boldsymbol{\theta}) = \begin{cases} -\log\left(\hat{p}\right) & \text{if } y = 1 \\ -\log\left(1 - \hat{p}\right) & \text{if } y = 0 \end{cases}$$

- Log-loss cost function (for <u>all instances</u>):

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} log\left(\hat{p}^{(i)}\right) + \left(1 - y^{(i)}\right) log\left(1 - \hat{p}^{(i)}\right)]$$

```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
log_reg.predict(..., ...)
```

Optimal weights are those that minimize ***J(θ)***

- Unfortunately, no analytical solution
- Must solve by <u>gradient descent</u> (iterative approach)